

MatrixPolicy: Role-Aware Optimization for Rational Transformer Nonlinearities

Anonymous authors
Paper under double-blind review

Abstract

Transformer nonlinear sublayers place large input and output matrices around a coordinatewise activation, but standard optimizers usually update these matrices without using their different functional roles. We introduce Rational Latent Basis (RLB), a grouped rational nonlinear sublayer that exposes normalized group coordinates, rational-curve gains, and a positive input/output rescaling structure. We then introduce MatrixPolicy, an optimizer that acts only on the RLB input and output matrices while leaving rational coefficients and the rest of the Transformer on AdamW. MatrixPolicy combines centered group-gradient redistribution, a role/depth-aware matrix update with a transient Muon branch, and bounded positive matrix-pair balancing. On fixed 12-layer, width-768 language-model pre-training runs over five corpora, MatrixPolicy obtains lower final validation loss than matched RLB and SiLU optimizer controls in both a 100M-token regime and a 300M-token regime. E2 token-to-target readouts show that these final-loss gains are accompanied by earlier arrival at representative validation-loss thresholds.

1 Introduction

Transformer nonlinear sublayers are usually described by their scalar activation: GELU, SiLU, SwiGLU, or a related gated variant (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). That shorthand hides a larger optimization object. A sublayer first chooses coordinates through an input matrix, applies a nonlinear response in those coordinates, and then recombines the resulting features through an output matrix. The two matrices do not play the same role, yet AdamW, Lion, Muon, and other general optimizers usually see them as ordinary tensors with no knowledge of which side of the nonlinearity they occupy.

This paper studies whether the nonlinear sublayer can expose enough structure for the optimizer to use. The key idea is to make the activation an interface, not only a pointwise function. If the sublayer provides normalized groups, a trainable rational response in each group, observable derivative and output scales, and a controlled positive rescaling relation between the adjacent matrices, then the optimizer can assign different update behavior to the input and output roles without changing the rest of the Transformer.

We instantiate this idea with Rational Latent Basis (RLB). RLB projects the residual stream into grouped latent coordinates, RMS-normalizes each group, applies one trainable real-pole-free P5/Q4 response per group, and projects back to the model width. The main variant used in this paper has no active local atoms: there are no trainable atom centers and no atom coefficient parameters. This choice keeps the nonlinear response simple enough to audit while preserving the signals that MatrixPolicy uses.

MatrixPolicy is the optimizer coupled to this sublayer. It is not a generic optimizer for every Transformer tensor. It acts only on the two RLB matrices, using batch-estimated rational gains, relative gradient scales, and the matrix pair ratio to redistribute group updates, select a transient early matrix-direction branch, and keep the input/output representatives balanced. Rational coefficients, attention weights, embeddings, and all other non-RLB-matrix parameters remain on AdamW.

We evaluate the method on a fixed 12-layer, width-768 causal Transformer across DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. The 100M-token regime is used to show early fixed-budget behavior, and the 300M-token regime tests whether the same ordering persists after three times as many tokens. Across both regimes, MatrixPolicy obtains the lowest aggregate validation loss among the matched controls. In the 300M-token regime, it reaches representative loss thresholds with fewer tokens than the fastest reachable non-MatrixPolicy comparator and the standard SiLU+AdamW baseline. The claims in this work are limited to this model scale and these token budgets.

The contributions are:

- a global-rational RLB sublayer that exposes matrix roles, curve-gain summaries, and a positive matrix-pair rescaling structure;
- MatrixPolicy, a role-aware optimizer for the RLB input and output matrices that leaves the rest of the Transformer on AdamW;
- a matched empirical package showing consistent fixed-budget gains over SiLU and global-rational RLB controls on five language-modeling corpora, together with E2 token-to-target readouts against the fastest reachable non-MatrixPolicy comparator and SiLU+AdamW.

2 Related Work

2.1 Nonlinear Transformer Sublayers

The original Transformer uses a positionwise nonlinear sublayer around self-attention (Vaswani et al., 2017). Later work improved the scalar response or gate, including GELU (Hendrycks & Gimpel, 2016), Swish/SiLU (Ramachandran et al., 2017), and GLU variants such as SwiGLU (Shazeer, 2020). RLB follows this line in replacing a fixed scalar response, but the target is different: it exposes optimizer-visible matrix roles around a grouped trainable response.

2.2 Rational Activations

Rational activations are attractive because low-degree rational functions can represent sharp transitions, saturation, and curved response profiles with few parameters. Recent approximation-theoretic work argues that trainable low-degree rational activations can have expressivity advantages over common fixed activations (Tang & Townsend, 2026). We use this result only as motivation for the activation family. RLB adds grouped RMS normalization and an optimizer-visible matrix interface, and MatrixPolicy adds role-aware optimization; these mechanisms are not consequences of the expressivity theory alone.

2.3 Pretraining Optimizers

Adam and AdamW remain standard choices for Transformer pretraining (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). One line of work changes the first-order update rule while retaining a generic tensor interface, including Lion and cautious optimizers (Chen et al., 2023; Liang et al., 2026). A second line uses matrix structure or low-rank information, including Muon, Sophia, GaLore, SOAP, and Adam-mini (Liu et al., 2025; 2024; Zhao et al., 2024; Vyas et al., 2025; Zhang et al., 2025). A third line reduces optimizer state or modifies the schedule, including CAME and schedule-free training (Luo et al., 2023; Defazio et al., 2024). MatrixPolicy is closest in spirit to matrix-aware optimization, but it does not apply one generic matrix rule to every second-order tensor. It uses the local structure of RLB to decide which matrices receive role-specific scaling and which activation statistics control their updates; the remaining Transformer parameters stay on AdamW. We refer to recent surveys for broader optimizer coverage (Wen et al., 2026).

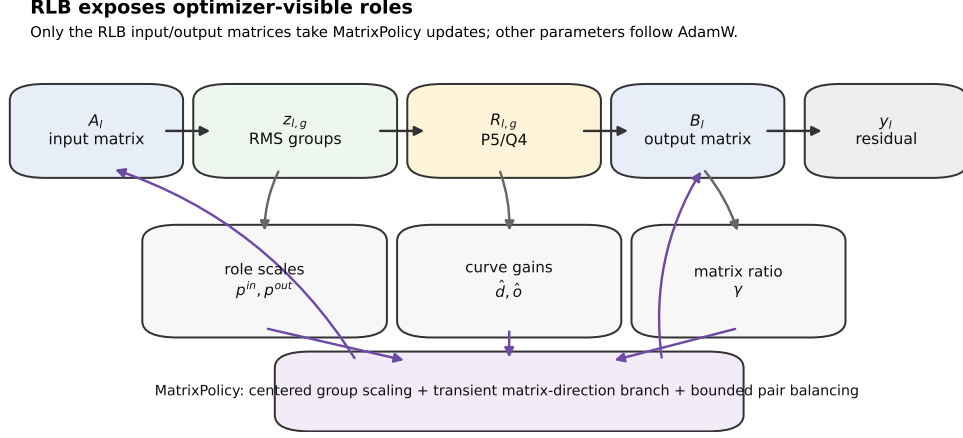


Figure 1: RLB exposes the input matrix, normalized rational groups, and output matrix as separate optimizer-visible objects. MatrixPolicy uses relative gradient scales, batch-estimated derivative/output gains, and the matrix-pair ratio to update only the RLB input/output matrices.

3 Method

RLB and MatrixPolicy are designed as an activation-optimizer pair. RLB defines the non-linear sublayer and exposes the statistics; MatrixPolicy consumes those statistics to update only the adjacent input and output matrices. Throughout the method we use

$$\text{clip}(x, a, b) = \min\{\max\{x, a\}, b\}, \quad (1)$$

$$\text{smoothstep}(a, b, x) = 3\omega^2 - 2\omega^3, \quad \omega = \text{clip}\left(\frac{x-a}{b-a}, 0, 1\right). \quad (2)$$

3.1 Rational Latent Basis Sublayer

For Transformer layer $l \in \{1, \dots, L\}$, let $x_l \in \mathbb{R}^d$ denote the input to the nonlinear sublayer. RLB first projects this input to a latent width $H = Gm$ and partitions the latent vector into G groups of width m :

$$z_l = A_l x_l, \quad z_l = \text{concat}_{g=1}^G z_{l,g}, \quad z_{l,g} \in \mathbb{R}^m, \quad (3)$$

where $A_l \in \mathbb{R}^{H \times d}$ is the input-domain matrix. Each group is normalized by its root-mean-square radius,

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon_{\text{rms}}}, \quad u_{l,g} = z_{l,g} / r_{l,g}, \quad (4)$$

with numerical floor $\epsilon_{\text{rms}} > 0$. A scalar response is applied coordinatewise in the normalized coordinates and then multiplied by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad h_l = \text{concat}_{g=1}^G h_{l,g}, \quad y_l = B_l h_l, \quad (5)$$

where $B_l \in \mathbb{R}^{d \times H}$ is the output-recombination matrix.

The curve $R_{l,g}$ is a trainable real-pole-free piecewise-rational P5/Q4 response,

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \quad P_{l,g}(u) = \sum_{i=0}^5 a_{l,g,i} u^i, \quad (6)$$

with denominator

$$Q_{l,g}(u) = 1 + |b_{l,g,1}| |u| + |b_{l,g,2}| u^2 + |b_{l,g,3}| |u|^3 + |b_{l,g,4}| u^4. \quad (7)$$

The absolute-value terms make the response piecewise rational rather than a single algebraic rational function of u , but they guarantee $Q_{l,g}(u) \geq 1$ on the real line. Numerator and denominator coefficients are initialized by fitting SiLU on $[-5, 5]$ and are trained thereafter. The implementation uses the same interval for probe-grid gain summaries. In the main variant there are no active local Cauchy atoms, trainable atom centers, or atom coefficient parameters.

The normalization in equation 4 gives the optimizer two views of each group. The normalized coordinate $u_{l,g}$ determines where the response is evaluated, while the radius $r_{l,g}$ carries group scale back to the residual stream. For fixed curves $R_l = \{R_{l,g}\}_{g=1}^G$, define the RLB block map

$$f_l(x; A_l, B_l, R_l) = B_l \text{concat}_{g=1}^G r_{l,g} R_{l,g}(u_{l,g}), \quad (8)$$

where $z_l = A_l x$, $z_{l,g}$, $r_{l,g}$, and $u_{l,g}$ are computed by equation 3–equation 4. In the idealized case $\epsilon_{\text{rms}} = 0$ with nonzero group radii, this construction admits the positive rescaling

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a)A_l, B_l D_l(a)^{-1}, R_l), \quad (9)$$

where $D_l(a) = \text{diag}(a_1 I_m, \dots, a_G I_m)$ for $a \in \mathbb{R}_{>0}^G$. MatrixPolicy uses this relation only for small, bounded balancing steps because the RMS floor and optimizer state make the identity approximate in the implemented system.

3.2 Role-Aware Optimizer Statistics

MatrixPolicy tracks role-specific relative gradient scales and curve-gain summaries for each layer and group. For any tensor V , define

$$\text{rms}(V) = \sqrt{\text{mean}(V^2)}, \quad \text{rms}_\epsilon(V) = \max\{\text{rms}(V), \epsilon\}. \quad (10)$$

Let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the row block of A_l , and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the column block of B_l . The relative matrix-gradient scales are

$$p_{l,g}^{\text{in}} = \frac{\text{rms}_{\epsilon_p}(\nabla A_{l,g})}{\text{rms}_{\epsilon_p}(A_{l,g})}, \quad p_{l,g}^{\text{out}} = \frac{\text{rms}_{\epsilon_p}(\nabla B_{l,g})}{\text{rms}_{\epsilon_p}(B_{l,g})}. \quad (11)$$

The coefficient-gradient scale for the global response is

$$p_{l,g}^R = \left[\frac{1}{2} (\text{mean}((\nabla a_{l,g,\cdot})^2) + \text{mean}((\nabla b_{l,g,\cdot})^2)) + \epsilon_p \right]^{1/2}. \quad (12)$$

MatrixPolicy maintains exponential moving averages $q_{l,g}^{\text{in}}$, $q_{l,g}^{\text{out}}$, and $q_{l,g}^R$ of these quantities with decay 0.95, and forms

$$\pi_{l,g} = \log q_{l,g}^{\text{in}} - \log q_{l,g}^{\text{out}}, \quad \alpha_{l,g} = \log q_{l,g}^R - \frac{1}{2} (\log q_{l,g}^{\text{in}} + \log q_{l,g}^{\text{out}}). \quad (13)$$

Here $\pi_{l,g}$ compares the update scale of the two adjacent matrix roles. The score $\alpha_{l,g}$ is a heuristic coefficient-activity statistic: because $p_{l,g}^R$ is a raw coefficient-gradient RMS rather than a dimensionless matrix pressure, MatrixPolicy uses $\alpha_{l,g}$ only inside bounded redistribution and gating rules.

The activation also provides batch-estimated curve gains. From a recent sample of normalized coordinates $u_{l,g}$ cached by the RLB forward path, MatrixPolicy uses floored summaries

$$\hat{d}_{l,g}^{\text{live}} = \max\{\text{rms}(R'_{l,g}(u_{l,g})), \epsilon_g\}, \quad \hat{o}_{l,g}^{\text{live}} = \max\{\text{rms}(R_{l,g}(u_{l,g})), \epsilon_g\}. \quad (14)$$

The derivative gain is a proxy for the input-domain role, and the output gain is a proxy for the scale seen by the recombination role. They are deliberately cheap summaries rather than full block-Jacobian preconditioners.

3.3 MatrixPolicy Update Rule

Let t be the optimizer step, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate

$$\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t) \quad (15)$$

turns on the group-gradient policy. For positive group values v_j , write $\text{geomean}_j v_j = \exp(G^{-1} \sum_{j=1}^G \log v_j)$. The multiplier for group g and role $r \in \{\text{in}, \text{out}\}$ is

$$c_{l,g,r} = c_{l,g,r}^{\text{gain}} c_{l,g,r}^{\text{pressure}} c_{l,g}^{\text{activity}}. \quad (16)$$

For the input role, the gain term uses $\hat{d}_{l,g}^{\text{live}}$; for the output role, it uses $\hat{o}_{l,g}^{\text{live}}$:

$$c_{l,g,\text{in}}^{\text{gain}} = \left(\frac{\text{geomean}_j \hat{d}_{l,j}^{\text{live}}}{\hat{d}_{l,g}^{\text{live}}} \right)^{0.20\phi_t}, \quad c_{l,g,\text{out}}^{\text{gain}} = \left(\frac{\text{geomean}_j \hat{o}_{l,j}^{\text{live}}}{\hat{o}_{l,g}^{\text{live}}} \right)^{0.20\phi_t}. \quad (17)$$

With $\tilde{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.50, 1.50)$, the relative-gradient term shifts update scale from the higher-scale role toward the role:

$$c_{l,g,\text{in}}^{\text{pressure}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \quad c_{l,g,\text{out}}^{\text{pressure}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \quad (18)$$

Coefficient-gradient scale damps both matrix roles when the rational coefficients are moving more strongly than the adjacent matrices:

$$c_{l,g}^{\text{activity}} = \exp \left[-0.20\phi_t \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right) \right]. \quad (19)$$

The final multiplier is recentered and clipped,

$$c_{l,g,r} \leftarrow \frac{c_{l,g,r}}{\text{geomean}_j c_{l,j,r}}, \quad c_{l,g,r} \leftarrow \text{clip}(c_{l,g,r}, 0.75, 1.35), \quad (20)$$

so the policy redistributes update scale across groups without changing the mean log multiplier before clipping.

The scaled matrix gradients are

$$\tilde{\nabla} A_{l,g} = c_{l,g,\text{in}} \nabla A_{l,g}, \quad \tilde{\nabla} B_{l,g} = c_{l,g,\text{out}} \nabla B_{l,g}. \quad (21)$$

After this scaling, the RLB matrices are updated by a role/depth AdamW branch and a transient Muon branch. Define $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The role factors are

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \quad \rho_{\text{out}}(l) = \text{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40), \quad (22)$$

and the AdamW matrix multiplier for role r is

$$s_{l,r}^{\text{Adam}} = \text{clip}(3.0 \max\{0.10, 1 + 1.20(\rho_r(l) - 1)\}, 0.40, 4.0). \quad (23)$$

The Muon branch is concentrated early in training and is reduced when relative scale imbalance or coefficient-gradient scale is high. Define

$$\Psi_{l,t} = \frac{1}{G} \sum_{g=1}^G |\text{clip}(\pi_{l,g}, -1.50, 1.50)|, \quad \Omega_{l,t} = \frac{1}{G} \sum_{g=1}^G \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right), \quad (24)$$

$$\psi_{l,t} = \text{clip}(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15). \quad (25)$$

The branch fraction is

$$\mu_{l,r,t} = \text{clip}(0.75 \text{smoothstep}(0.02, 0.12, \xi_t)[1 - \text{smoothstep}(0.20, 0.36, \xi_t)]\rho_r(l)\psi_{l,t}, 0, 0.75). \quad (26)$$

For $M_{l,\text{in}} = A_l$ and $M_{l,\text{out}} = B_l$, MatrixPolicy composes an AdamW matrix step with the Muon branch:

$$\begin{aligned}\widetilde{M}_{l,r}^{t+1} &= \text{AdamW}_{\eta_t s_{l,r}^{\text{Adam}}(1-\mu_{l,r,t})}(M_{l,r}^t; \widetilde{\nabla} M_{l,r}^t), \\ M_{l,r}^{t+1} &= \text{Muon}_{\eta_t \mu_{l,r,t}}(\widetilde{M}_{l,r}^{t+1}; \widetilde{\nabla} M_{l,r}^t),\end{aligned}\quad (27)$$

Equation 27 is operational update notation: AdamW and Muon carry their own optimizer states, weight-decay and momentum terms are handled by those optimizers, and both substeps use the gradient from training step t . Here η_t is the base learning rate and Muon denotes the torch.optim.Muon update on two-dimensional RLB matrix parameters: momentum 0.95, five Newton–Schulz orthogonalization steps, and the match-RMS learning-rate adjustment of the Muon optimizer (Liu et al., 2025). When all Muon fractions have decayed to zero, this branch is inactive.

The last operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated every five optimizer steps; on other steps it is skipped. Let $\mathcal{U} = \{u_k\}_{k=1}^{129}$ be a uniform probe grid over $[-5, 5]$, and define $\text{rms}_{\mathcal{U}}(\varphi) = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} \varphi(u_k)^2}$. The probe gains are

$$\hat{d}_{l,g}^{\text{probe}} = \max\{\text{rms}_{\mathcal{U}}(R'_{l,g}), \epsilon_g\}, \quad \hat{o}_{l,g}^{\text{probe}} = \max\{\text{rms}_{\mathcal{U}}(R_{l,g}), \epsilon_g\}, \quad (28)$$

and the current matrix log ratio is computed with floored matrix RMS values,

$$\gamma_{l,g} = \log \text{rms}_{\epsilon_p}(A_{l,g}) - \log \text{rms}_{\epsilon_p}(B_{l,g}). \quad (29)$$

The target ratio combines curve-gain and relative-gradient terms,

$$\tau_{l,g} = \frac{1}{2} \left(\log \hat{d}_{l,g}^{\text{probe}} - \log \hat{o}_{l,g}^{\text{probe}} \right) + 0.25 \bar{\pi}_{l,g}, \quad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \quad (30)$$

Coefficient-gradient scale modulates the rescaling strength through

$$a_{l,g}^{\text{rs}} = \exp(0.10 \bar{\alpha}_{l,g}), \quad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \quad (31)$$

With schedule

$$s_{l,t} = 0.50 \text{ smoothstep}(0.03, 0.35, \xi_t) \text{ clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \quad (32)$$

the active-step log move is

$$\ell_{l,g,t} = \text{clip} \left(\frac{1}{2} [\tau_{l,g} - \gamma_{l,g}] s_{l,t} a_{l,g}^{\text{rs}}, -0.030, 0.030 \right). \quad (33)$$

The matrix blocks are then updated as

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \quad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \quad (34)$$

4 Experiments

4.1 Setup

The empirical study uses a 12-layer, width-768 causal Transformer with sequence length 256, 12 attention heads, latent width 3072 for RLB, and 32768 global tokens per optimizer step. We evaluate two regimes: E1 trains for 3050 steps (approximately 100M tokens), and E2 trains for 9150 steps (approximately 300M tokens). Each method is run with three seeds on DCLM/DataComp-LM (Li et al., 2024), FineWeb-Edu and FineWeb (Penedo et al., 2024), Dolma-sample (Soldaini et al., 2024), and C4 (Raffel et al., 2020).

MatrixPolicy is compared with matched global-rational RLB controls using AdamW, Lion, Muon, SOAP, CAME, and ScheduleFree-style optimizers, and with SiLU controls under the same model and token budgets. The main tables report aggregate validation losses over the shared three-seed protocol.

MatrixPolicy update in one optimizer step.

1. Update moving averages of matrix-gradient scales and coefficient-gradient scales from current gradients.
2. Read cached batch-estimated curve gains from the most recent RLB forward path.
3. Step non-matrix parameters, including rational coefficients, with AdamW.
4. Scale RLB matrix gradients with the centered group multipliers in equation 20.
5. Step RLB matrices with the AdamW matrix branch and any active Muon branch in equation 27.
6. Every five optimizer steps, apply the bounded positive matrix-pair rescaling in equation 34.

Figure 2: MatrixPolicy affects only the input and output matrices of RLB sublayers. All other parameters use the AdamW path.

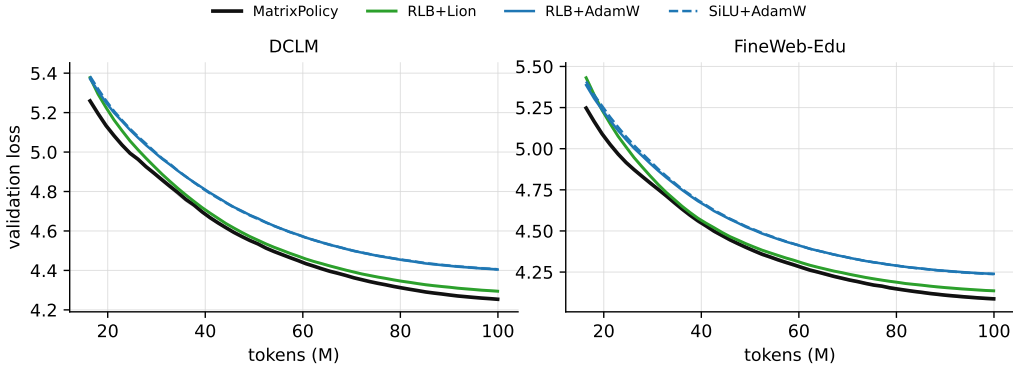


Figure 3: Representative E1 validation-loss trajectories reconstructed from raw JSONL evaluation events. Lines are seed means and bands are one sample standard deviation over three seeds.

4.2 Validation Trajectories

Figure 3 shows representative E1 validation trajectories on DCLM and FineWeb-Edu. The curves are means over three seeds and the bands are sample standard deviations at the native 50-step evaluation cadence. MatrixPolicy separates early from RLB+AdamW and SiLU+AdamW, and remains below the strongest shown RLB control, RLB+Lion, by the end of the 100M-token budget. The point of the figure is not that these two datasets are special; they illustrate the same ordering summarized across all five datasets in Table 1.

4.3 Final Losses Across Datasets

Table 1 reports the final validation loss for MatrixPolicy and the strongest non-MatrixPolicy aggregate comparator in each dataset and regime. MatrixPolicy is the lowest-loss method in every E1 and E2 dataset shown. The strongest non-MatrixPolicy comparator is RLB+Lion in all E1 datasets. In E2 it is RLB+Lion except on FineWeb-Edu, where RLB+Muon is the strongest aggregate comparator.

Table 2 broadens the comparison beyond the best non-MatrixPolicy row. The entries are average final-loss gaps relative to MatrixPolicy across the five datasets; positive values mean MatrixPolicy is lower. This table separates two effects. First, RLB by itself is not enough: RLB+AdamW is close to SiLU+AdamW. Second, MatrixPolicy improves over both the standard baseline and the strongest RLB optimizer controls.

Table 1: Final validation loss under the fixed model protocol. Values are mean $\pm$ sample standard deviation over three seeds; lower is better. MP denotes MatrixPolicy.

Dataset	E1 MP	E1 best non-MP	E1 gap	E2 MP	E2 best non-MP	E2 gap
DCLM	4.2538 $\pm$ 0.0063	RLB+Lion 4.2946 $\pm$ 0.0083	0.0408	3.9518 $\pm$ 0.0282	RLB+Lion 3.9887 $\pm$ 0.0295	0.0369
FineWeb-Edu	4.0873 $\pm$ 0.0102	RLB+Lion 4.1361 $\pm$ 0.0083	0.0488	3.7015 $\pm$ 0.0212	RLB+Muon 3.7373 $\pm$ 0.0187	0.0358
FineWeb	4.3162 $\pm$ 0.0125	RLB+Lion 4.3626 $\pm$ 0.0112	0.0464	3.9623 $\pm$ 0.0081	RLB+Lion 3.9960 $\pm$ 0.0105	0.0337
Dolma-sample	4.3253 $\pm$ 0.0053	RLB+Lion 4.3622 $\pm$ 0.0066	0.0369	3.8062 $\pm$ 0.0073	RLB+Lion 3.8412 $\pm$ 0.0085	0.0351
C4	4.2837 $\pm$ 0.0197	RLB+Lion 4.3271 $\pm$ 0.0160	0.0434	3.8777 $\pm$ 0.0144	RLB+Lion 3.9132 $\pm$ 0.0139	0.0355

Table 2: Final-loss gaps relative to MatrixPolicy across the five datasets. Positive gaps mean MatrixPolicy has lower loss; the worst column is the smallest per-dataset gap for that comparator.

Comparator	E1 mean	E1 worst	E2 mean	E2 worst
RLB+Lion	0.043	0.037	0.036	0.034
SiLU+Lion	0.065	0.062	0.042	0.039
RLB+Muon	0.205	0.193	0.039	0.036
SiLU+Muon	0.203	0.191	0.047	0.044
RLB+AdamW	0.156	0.151	0.099	0.098
SiLU+AdamW	0.157	0.150	0.100	0.098
RLB+SOAP	0.220	0.162	0.125	0.109
SiLU+SOAP	0.171	0.162	0.153	0.145
RLB+ScheduleFree	0.687	0.633	0.422	0.406
SiLU+ScheduleFree	0.706	0.649	0.430	0.409
RLB+CAME	0.812	0.760	0.498	0.460
SiLU+CAME	0.817	0.757	0.441	0.416

4.4 Token-to-Target Readout

Endpoint losses do not show whether a method reaches useful validation levels earlier. Table 3 reports E2 token-to-target readouts at representative thresholds. The first comparator view uses the fastest non-MatrixPolicy method that reaches the threshold on the shared seeds. The second view uses the standard SiLU+AdamW baseline at thresholds where all three common seeds reach the target. Token counts are quantized by the native 50-step validation cadence, corresponding to 1.64M tokens.

4.5 Scope of the Evidence

The E2 gaps are smaller than the E1 gaps, but they remain positive across the five datasets and across the comparator families in Table 2. The evidence does not establish behavior for larger model sizes, longer token budgets, or different architectural families.

5 Limitations and Larger-Scale Evaluation

The current evidence stops at a 12-layer, width-768 model and a 300M-token budget. We therefore do not infer larger-scale behavior from these runs. When larger experiments are complete, this section should be replaced by their results; until then, the claims are restricted to the model scale, corpora, and token budgets reported above.

6 Conclusion

RLB turns the nonlinear Transformer sublayer into an optimizer-visible interface: it exposes grouped normalized coordinates, trainable rational responses, and a positive matrix-pair structure. MatrixPolicy uses that interface to update the RLB input and output matrices with role-aware statistics while leaving the rest of the model on AdamW. The fixed-scale language-model experiments show that the optimizer design, not merely the rational ac-

Table 3: Representative E2 token-to-target readouts. Entries show target loss: Matrix-Policy tokens $\rightarrow$ comparator tokens, followed by saved tokens and saved fraction. The SiLU+AdamW targets use thresholds reached by all three shared seeds.

Dataset	Fastest non-MP target	Saved	Saved %	SiLU+AdamW target	Saved / saved %
DCLM	4.05 : 201.0M $\rightarrow$ 220.1M	19.1M	8.7	4.10 : 174.8M $\rightarrow$ 227.7M	53.0M / 23.3
FineWeb-Edu	3.80 : 209.7M $\rightarrow$ 223.4M	13.7M	6.1	3.85 : 183.0M $\rightarrow$ 237.0M	54.1M / 22.8
FineWeb	4.05 : 213.0M $\rightarrow$ 228.8M	15.8M	6.9	4.10 : 185.7M $\rightarrow$ 241.4M	55.7M / 23.1
Dolma-sample	3.85 : 242.5M $\rightarrow$ 280.2M	37.7M	13.5	3.95 : 180.8M $\rightarrow$ 232.7M	51.9M / 22.3
C4	4.00 : 195.0M $\rightarrow$ 204.8M	9.8M	4.8	4.00 : 195.0M $\rightarrow$ 264.9M	69.9M / 26.4

tivation family, is responsible for the observed final-loss and token-to-target gains in the evaluated regimes.

References

- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL <https://neurips.cc/virtual/2024/poster/96925>.
- Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). arXiv preprint arXiv:1606.08415, 2016. URL <https://arxiv.org/abs/1606.08415>.
- Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL <https://mlanthology.org/iclr/2015/kingma2015iclr-adam/>.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zBPZeRjfgu>.
- Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. arXiv preprint arXiv:2502.16982, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.

- Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL <https://aclanthology.org/2023.acl-long.243/>.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140): 1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. arXiv preprint arXiv:1710.05941, 2017. URL <https://arxiv.org/abs/1710.05941>.
- Noam Shazeer. GLU Variants Improve Transformer. arXiv preprint arXiv:2002.05202, 2020. URL <https://arxiv.org/abs/2002.05202>.
- Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. arXiv preprint arXiv:2402.00159, 2024. URL <https://arxiv.org/abs/2402.00159>.
- Maosen Tang and Alex Townsend. Rational Neural Networks have Expressivity Advantages. arXiv preprint arXiv:2602.12390, 2026. doi: 10.48550/arXiv.2602.12390. URL <https://arxiv.org/abs/2602.12390>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In Advances in Neural Information Processing Systems, 2017. URL https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.
- Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In International Conference on Learning Representations, 2026. URL <https://openreview.net/forum?id=2J51qUZ0iG>.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In International Conference on Machine Learning, 2024. URL <https://openreview.net/forum?id=hYHsrKDiX7>.

A RLB Calculus and Matrix-Role Structure

A.1 Pole-free global response

For one layer and group, write

$$P(u) = \sum_{i=0}^5 a_i u^i, \quad Q(u) = 1 + \sum_{j=1}^4 |b_j| \phi_j(u), \quad \phi(u) = (|u|, u^2, |u|^3, u^4). \quad (35)$$

Since $\phi_j(u) \geq 0$ for all real u , $Q(u) \geq 1$ and $R(u) = P(u)/Q(u)$ has no real pole. Away from the nondifferentiable point of the absolute-value term,

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \quad (36)$$

where

$$P'(u) = \sum_{i=1}^5 i a_i u^{i-1}, \quad Q'(u) = |b_1| \text{sign}(u) + 2|b_2|u + 3|b_3|u|u| + 4|b_4|u^3. \quad (37)$$

The implementation uses the autodiff convention $\text{sign}(0) = 0$ for derivative-gain statistics. The coefficient features are explicit:

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \quad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2} \text{sign}(b_j) \phi_j(u), \quad (38)$$

with the usual subgradient convention at zero. These derivatives make output and derivative gains directly observable from the shared global curve, without introducing local-atom parameters.

A.2 Normalized-group Jacobian and gain summaries

For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1} \|z\|_2^2 + \epsilon_{\text{rms}}}, \quad u(z) = z/r(z), \quad h(z) = r(z)R(u(z)), \quad (39)$$

where R is applied coordinatewise. The radius derivative is

$$dr = \frac{z^\top dz}{mr} = \frac{u^\top dz}{m}, \quad (40)$$

and the normalized-coordinate derivative is

$$du = \frac{1}{r} \left(I - \frac{uu^\top}{m} \right) dz. \quad (41)$$

Therefore the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u) \frac{u^\top}{m} + \text{diag}(R'(u)) \left(I - \frac{uu^\top}{m} \right). \quad (42)$$

This decomposition motivates separate output-gain and derivative-gain summaries. The vector $R(u)$ controls the direct feature scale seen by B_l , while $R'(u)$ appears in the tangent component of the input Jacobian seen by A_l . The summary $\hat{d}$ is not the full input Jacobian: the full input gradient also depends on the radial $R(u)u^\top/m$ term and on $B_{l,g}$.

A.3 Positive matrix-pair rescaling with an RMS floor

Set $\epsilon_{\text{rms}} = 0$ and take $a > 0$. Then

$$r(az) = ar(z), \quad u(az) = u(z), \quad h(az) = ah(z). \quad (43)$$

Scaling $A_{l,g}$ by a and the matching columns of $B_{l,g}$ by a^{-1} therefore leaves the group contribution unchanged. Applying this independently to all groups gives equation 9.

With $\epsilon_{\text{rms}} > 0$, define r_ϵ and u_ϵ by equation 4. For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(az) = \kappa(a, z)u_\epsilon(z), \quad \kappa(a, z) = \frac{a\sqrt{\rho^2 + \epsilon_{\text{rms}}}}{\sqrt{a^2\rho^2 + \epsilon_{\text{rms}}}}. \quad (44)$$

Let L_R be a Lipschitz constant of the coordinatewise curve on the segment between $u_\epsilon(z)$ and $\kappa(a, z)u_\epsilon(z)$. After compensating the output matrix by a^{-1} , the group-feature error obeys

$$\|a^{-1}h_\epsilon(az) - h_\epsilon(z)\|_2 \leq \left| \frac{r_\epsilon(az)}{a} - r_\epsilon(z) \right| \|R(u_\epsilon(z))\|_2 \quad (45)$$

$$+ \frac{r_\epsilon(az)}{a} L_R |\kappa(a, z) - 1| \|u_\epsilon(z)\|_2. \quad (46)$$

Both terms vanish when $\epsilon_{\text{rms}} = 0$. When $\rho^2 \gg \epsilon_{\text{rms}}$ and the local response and slope are bounded on the segment between $u_\epsilon(z)$ and $\kappa(a, z)u_\epsilon(z)$, the bound is small; near the floor, the approximation can be poor. This is the reason MatrixPolicy uses clipped rescaling moves rather than treating matrix-pair balancing as an exact function-preserving operation.

B MatrixPolicy Design Rationale

B.1 Role-specific gradients and gain selection

For one training example, let $\bar{y}_l = \nabla_{y_l} \mathcal{L}$ be the upstream gradient, and let $J_{l,g} = \partial h_{l,g} / \partial z_{l,g}$ be the group Jacobian in equation 42. The two matrix gradients have different forms:

$$\nabla_{B_{l,g}} \mathcal{L} = \bar{y}_l h_{l,g}^\top, \quad \nabla_{A_{l,g}} \mathcal{L} = (J_{l,g}^\top B_{l,g}^\top \bar{y}_l) x_l^\top. \quad (47)$$

The output matrix sees the realized feature vector $h_{l,g}$ directly. The input matrix sees the upstream gradient filtered through $B_{l,g}$ and the RLB group Jacobian. This motivates using output-gain summaries for the output role and derivative-gain summaries for the input role, while recognizing that these summaries are proxies rather than full matrix preconditioners.

B.2 Scale-normalized pressure and centered redistribution

The exact derivative along the homogeneous rescaling direction would be

$$\Delta_g = \langle \nabla_{A_{l,g}} \mathcal{L}, A_{l,g} \rangle_F - \langle \nabla_{B_{l,g}} \mathcal{L}, B_{l,g} \rangle_F. \quad (48)$$

MatrixPolicy instead uses RMS relative gradient scales because they are stable proxies for how strongly each matrix role is being moved. For nonzero unfloored W , if $W^+ = W - \eta \mathcal{G}$ and η is small, then

$$\log \text{rms}(W^+) - \log \text{rms}(W) = -\eta \frac{\langle \mathcal{G}, W \rangle_F}{\|W\|_F^2} + O(\eta^2), \quad (49)$$

and Cauchy-Schwarz gives

$$\left| \frac{\langle \mathcal{G}, W \rangle_F}{\|W\|_F^2} \right| \leq \frac{\|\mathcal{G}\|_F}{\|W\|_F} = \frac{\text{rms}(\mathcal{G})}{\text{rms}(W)}. \quad (50)$$

The implemented pressures are floored versions of this stable proxy, not natural gradient terms.

Before clipping, the group policy is recentered by

$$\hat{c}_{l,g,r} = \frac{c_{l,g,r}}{\text{geomean}_j c_{l,j,r}}. \quad (51)$$

Therefore

$$\frac{1}{G} \sum_{g=1}^G \log \hat{c}_{l,g,r} = \frac{1}{G} \sum_{g=1}^G \log c_{l,g,r} - \log (\text{geomean}_j c_{l,j,r}) = 0. \quad (52)$$

The centered policy redistributes group update scale while preserving the mean log multiplier before clipping. Clipping prevents noisy group statistics from dominating a matrix update, at the cost of slightly perturbing the exact zero-mean log property.

B.3 Bounded matrix-pair balancing and transient direction updates

Let

$$\gamma_{l,g} = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (53)$$

On an active positive-rescaling step, $A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}$, so the log ratio changes as

$$\gamma_{l,g}^+ = \gamma_{l,g} + 2\ell_{l,g,t}. \quad (54)$$

Ignoring clipping and writing $\lambda_{l,g,t} = s_{l,t} a_{l,g}^{\text{rs}}$, the update in equation 33 gives

$$\gamma_{l,g}^+ = (1 - \lambda_{l,g,t})\gamma_{l,g} + \lambda_{l,g,t}\tau_{l,g}. \quad (55)$$

Thus, in the $\epsilon_{\text{rms}} = 0$ and unclipped idealization, each active rescaling step moves the matrix representative toward the target ratio without changing the represented homogeneous block map. In the implemented setting this operation runs every five optimizer steps, $\lambda_{l,g,t}$ is bounded by the update schedule and activity factor, and the actual log move is clipped by 0.030.

The transient Muon branch uses the window

$$w(\xi_t) = \text{smoothstep}(0.02, 0.12, \xi_t) [1 - \text{smoothstep}(0.20, 0.36, \xi_t)]. \quad (56)$$

This confines the matrix-direction branch to the phase in which changes to latent directions most directly affect the coordinates seen by the rational curves. The coefficient-gradient penalty $\psi_{l,t}$ further reduces the branch when the rational coefficients are already moving strongly. The design therefore keeps the branch tied to the state of the RLB sublayer rather than applying a persistent generic matrix rule throughout training.